CHATGPT ASSIGNMENT CONVERSATION
Project: AI-Assisted Box Selection System
Technology: Python, Django, Django REST Framework

USER:
My development setup: Editor: vscode. Python installed: yes. GitHub account: yes. Let's start building the Django REST Framework assignment step by step.

ASSISTANT:
Perfect. We'll build the project using VS Code + Python + Django REST Framework + GitHub.

Since this is a hiring assignment, I want you to understand each component and be able to explain your implementation to Tradexa's technical interviewer.

We'll build a working backend, test it locally, and prepare the submission files.

Step 1 — Create your Django project

1. Create a project folder
Open VS Code, select File → Open Folder, and create a folder named:

box-selection-system

Open the VS Code terminal:
Terminal → New Terminal

2. Create a virtual environment
python -m venv venv

Activate it on Windows:
venv\Scripts\activate

3. Install dependencies
python -m pip install django djangorestframework

4. Create the Django project
django-admin startproject config .
python manage.py startapp box_selection

5. Register the application in config/settings.py:
'rest_framework',
'box_selection',

6. Run migrations:
python manage.py migrate

Then start the server:
python manage.py runserver

USER:
yarr how to do this i am not getting re i m very new:
Step 1 — Create your Django project
Starting development
1. Create a project folder
Open VS Code, select File → Open Folder, and create a folder named:
box-selection-system
Open the VS Code terminal:
Terminal → New Terminal

ASSISTANT:
Don't worry. We'll do only one small task at a time.

Open VS Code.
Click File → Open Folder.
Create a folder named box-selection-system.
Select the folder.
Open Terminal → New Terminal.

USER:
next

ASSISTANT:
Step 2: Create a Python virtual environment.

Run:
python -m venv venv

Activate it:
.\venv\Scripts\Activate.ps1

You should see (venv) at the beginning of your terminal prompt.

USER:
(venv) PS C:\Users\lkada\box-selection-system> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
November 05, 2026 - 11:47:58
Django version 6.1.3, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting.
[05/Nov/2026 11:49:05] "GET / HTTP/1.1" 200 12149
Not Found: /favicon.ico
[05/Nov/2026 11:49:07] "GET /favicon.ico HTTP/1.1" 404 2280
[05/Nov/2026 11:49:15] "GET / HTTP/1.1" 200 12149

ASSISTANT:
Your Django project is working. The GET / HTTP/1.1 200 response confirms that the browser successfully connected to Django. The favicon.ico 404 is harmless.

USER:
working now.. next

ASSISTANT:
Step 7 — Create database models.

The system needs four models:
Product — product name, dimensions and weight.
ShippingBox — box dimensions, weight capacity and cost.
Order — customer order information.
OrderItem — products and quantities in each order.

Open box_selection/models.py and add the four models.

USER:
yup done:
(venv) PS C:\Users\lkada\box-selection-system> python manage.py migrate
Operations to perform:
Apply all migrations: admin, auth, box_selection, contenttypes, sessions
Running migrations:
Applying box_selection.0001_initial... OK

ASSISTANT:
All four models have been successfully created in your database.

Step 8 — Register models in Django Admin.

Open box_selection/admin.py and add:

```
from django.contrib import admin
from .models import Product, ShippingBox, Order, OrderItem

admin.site.register(Product)
admin.site.register(ShippingBox)
admin.site.register(Order)
admin.site.register(OrderItem)
```

Create a superuser:
```
python manage.py createsuperuser
```

Start the server:
```
python manage.py runserver
```

Open:
http://127.0.0.1:8000/admin/

USER:
Django administration

Welcome, laxmi. View site / Change password / Log out

Site administration

ASSISTANT:
Excellent! You've successfully logged into Django Admin.

Step 9 — Add sample products and shipping boxes.

Products:
Laptop: 30 × 20 × 3 cm, 2 kg
Mouse: 10 × 6 × 4 cm, 0.2 kg
Keyboard: 35 × 12 × 3 cm, 0.8 kg

Shipping boxes:
Small: 35 × 25 × 10 cm, max weight 3 kg, cost 20
Medium: 45 × 35 × 20 cm, max weight 10 kg, cost 35
Large: 60 × 45 × 30 cm, max weight 20 kg, cost 60

USER:
Django administration
Welcome, laxmi.
Site administration
Box_Selection
Order items
Recent actions:
Added Shipping box
Added Shipping box
Added Shipping box
Added Product
Added Product
Added Product

ASSISTANT:
Perfect! You've successfully added all 3 products and 3 shipping boxes.

Next step: Build the recommendation API.

We'll create the code that:

- Accepts an order containing products and quantities.
- Checks whether products fit inside a shipping box, allowing rotation.
- Checks total weight against the box's maximum capacity.
- Recommends the lowest-cost suitable box.

Create a new file:
services.py

Test Execution Output

Test Command

python manage.py test box_selection

Actual Test Results

Found 4 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
....

Ran 4 tests in 0.013s

OK
Destroying test database for alias 'default'...

Summary
Total tests: 4
Passed: 4
Failed: 0
System check errors: 0

Result: All four automated tests passed successfully.

ASSISTANT:
Open TEST_OUTPUT.md, press Ctrl + A, and paste the complete clean version.

Test Execution Output

1. Test Command

The following command was executed in the VS Code terminal:

python manage.py test box_selection

2. Actual Test Results

Found 4 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
....

Ran 4 tests in 0.013s

OK
Destroying test database for alias 'default'...

3. Test Summary

Total test cases: 4
Passed: 4
Failed: 0
System check errors: 0

4. Conclusion

All four automated test cases passed successfully.

Overall Result: PASS

USER:
donr

ASSISTANT:
Perfect! TEST_OUTPUT.md is complete.